

AIRBNB DATABASE SYSTEM - COMPLETE IMPLEMENTATION

COURSE: DLBDSPBDM01 - BUILD A DATA MART IN SQL

NAME: IPADEOLA OLUWATOYIN ENIOLA

MATRICULATION NUMBER: 92130758

DATE: 19/03/2026

AIRBNB DATABASE SYSTEM IMPLEMENTATION

COMPLETE DATA MART SOLUTION

Project Scope:

- ▶ Design and implement a relational database for Airbnb-style platform
- ▶ Support 4 user roles: Guest, Host, Admin, Support
- ▶ Manage properties, bookings, payments, reviews, and communications
- ▶ Implement triple relationships and recursive relationships
- ▶ Create advanced database objects (views, procedures, functions, triggers)

Key Achievements:

- ▶ 22 normalized tables with referential integrity
- ▶ 20+ sample data rows per main table
- ▶ 3 triple relationships implemented
- ▶ 2 recursive relationships implemented
- ▶ Comprehensive business logic through stored procedures

PROBLEM STATEMENT & REQUIREMENTS

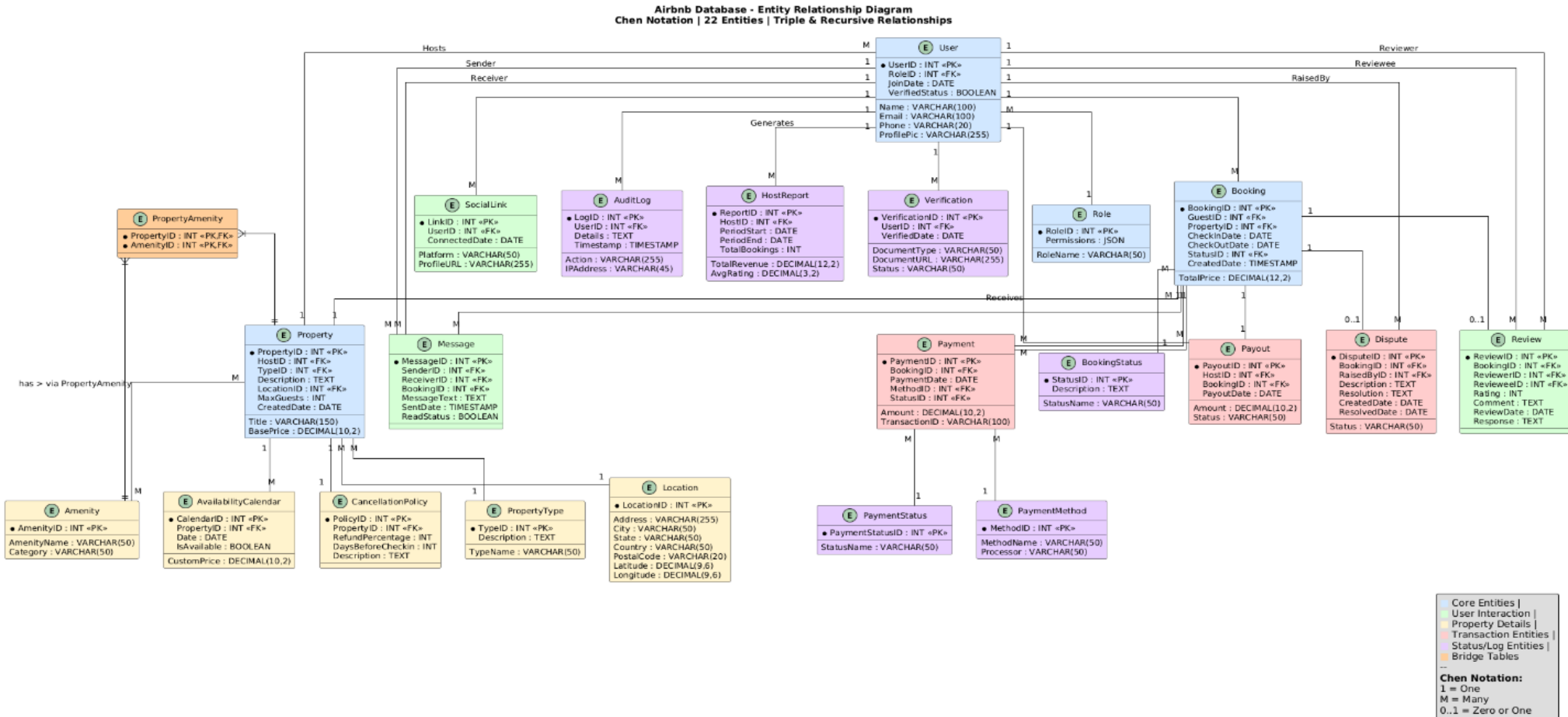
Business Problem

- ▶ Modern property rental platforms face challenges with:
- ▶ **Data fragmentation** across multiple systems
- ▶ **Inconsistent information** about properties and users
- ▶ **Poor scalability** as user base grows
- ▶ **Limited analytics** capabilities
- ▶ **Manual processes** for bookings and payments

Solution Requirements

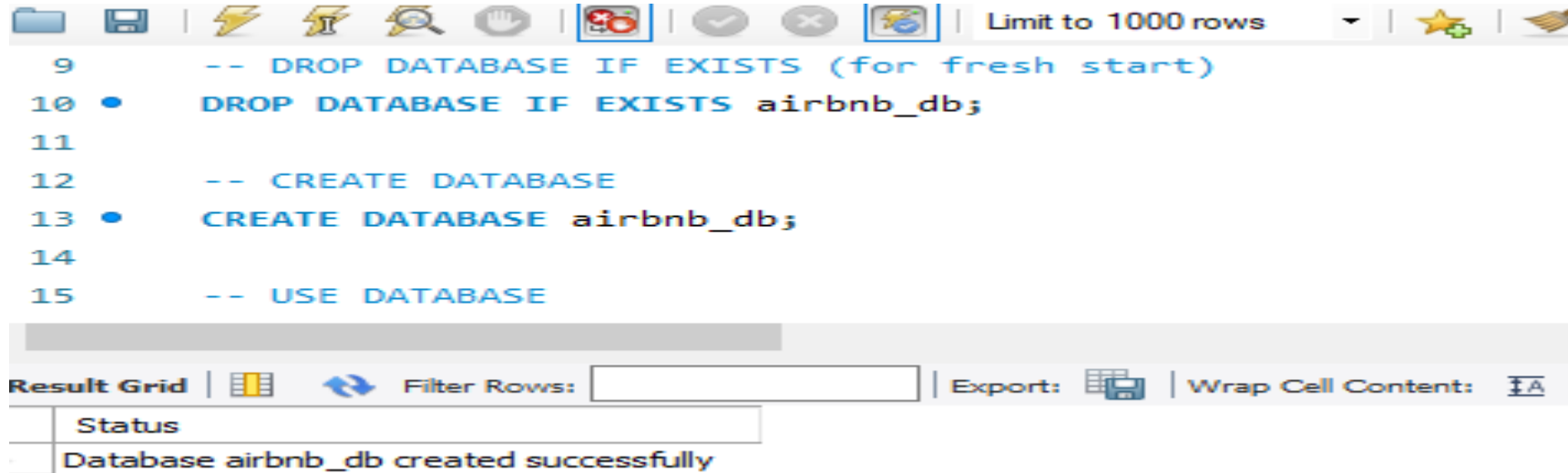
- ▶ Centralized relational database
- ▶ Role-based access control (Guest, Host, Admin, Support)
- ▶ Property listing and management system
- ▶ Booking and payment processing
- ▶ Review and rating system
- ▶ Communication platform
- ▶ Reporting and analytics

ER MODEL: GRAPHICAL REPRESENTATION OF THE AIRBNB DATABASE



DATABASE CREATION

DATABASE SETUP



The screenshot shows a database management tool interface. At the top, there is a toolbar with various icons including a folder, save, lightning bolt, magnifying glass, hand, and a red 'X' icon. Below the toolbar, a SQL script is displayed with line numbers 9 through 15. The script contains the following commands:

```
9      -- DROP DATABASE IF EXISTS (for fresh start)
10 •   DROP DATABASE IF EXISTS airbnb_db;
11
12      -- CREATE DATABASE
13 •   CREATE DATABASE airbnb_db;
14
15      -- USE DATABASE
```

Below the script, there is a 'Result Grid' section. It includes a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' checkbox. The result grid shows a single row with the status 'Database airbnb_db created successfully'.

Key Features:

- ▶ Clean installation with DROP IF EXISTS
- ▶ UTF-8 character set for international support
- ▶ InnoDB engine for transaction support
- ▶ Proper isolation for testing

ROLE TABLE

CREATE ROLE TABLE STATEMENT

```
6  -- 1. ROLE TABLE
7  • CREATE TABLE Role (
8      RoleID INT PRIMARY KEY AUTO_INCREMENT,
9      RoleName VARCHAR(50) NOT NULL,
10     Permissions JSON,
11     CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
12     UNIQUE(RoleName)
13 );
14
```

Total Tables Created
22

INSERT SAMPLE DATA

```
6  -- 1. Insert Roles (no dependencies)
7  • INSERT INTO Role (RoleName, Permissions) VALUES
8      ('Guest', '{"book": true, "review": true, "message": true}'),
9      ('Host', '{"list_property": true, "manage_bookings": true, "receive_payments": true}'),
10     ('Admin', '{"manage_users": true, "view_reports": true, "resolve_disputes": true}'),
11     ('Support', '{"view_tickets": true, "assist_users": true}');
12
13 • SELECT '4 roles inserted' AS Status;
14
15 -- 2. Insert Users (depends on Role table)
```

Users	Properties	Bookings	Payments	Reviews	Messages
24	22	22	22	14	12

TEST QUERY & RESULTS

```
1 • SELECT * FROM ROLE;
```

RoleID	RoleName	Permissions	CreatedAt
1	Guest	{"book": true, "review": true, "message": true}	2026-03-07 12:00:46
2	Host	{"list_property": true, "manage_bookings": true, "receive_payments": true}	2026-03-07 12:00:46
3	Admin	{"manage_users": true, "view_reports": true, "resolve_disputes": true}	2026-03-07 12:00:46
4	Support	{"assist_users": true, "view_tickets": true}	2026-03-07 12:00:46

USER TABLE

CREATE USER TABLE STATEMENT

```
16 • CREATE TABLE User (  
17     UserID INT PRIMARY KEY AUTO_INCREMENT,  
18     RoleID INT NOT NULL,  
19     FirstName VARCHAR(50) NOT NULL,  
20     LastName VARCHAR(50) NOT NULL,  
21     Email VARCHAR(100) NOT NULL UNIQUE,  
22     Phone VARCHAR(20),  
23     ProfilePic VARCHAR(255),  
24     PasswordHash VARCHAR(255) NOT NULL,  
25     JoinDate DATE DEFAULT (CURRENT_DATE),  
26     VerifiedStatus BOOLEAN DEFAULT FALSE,  
27     LastLogin TIMESTAMP NULL,  
28     CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
29     UpdatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
30     FOREIGN KEY (RoleID) REFERENCES Role(RoleID) ON DELETE RESTRICT,  
31     INDEX idx_email (Email),  
32     INDEX idx_role (RoleID)  
);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Total Tables Created
22

INSERT SAMPLE DATA

```
15 -- 2. Insert Users (depends on Role table)  
16 • INSERT INTO User (RoleID, FirstName, LastName, Email, Phone, PasswordHash, VerifiedStatus) VALUES  
17 (1, 'John', 'Smith', 'john.smith@email.com', '+1-555-0101', 'hashed_password_1', TRUE),  
18 (1, 'Emma', 'Johnson', 'emma.j@email.com', '+1-555-0102', 'hashed_password_2', TRUE),  
19 (1, 'Michael', 'Brown', 'michael.b@email.com', '+1-555-0103', 'hashed_password_3', FALSE),  
20 (1, 'Sarah', 'Davis', 'sarah.d@email.com', '+1-555-0104', 'hashed_password_4', TRUE),  
21 (1, 'David', 'Wilson', 'david.w@email.com', '+1-555-0105', 'hashed_password_5', TRUE),  
22 (2, 'Robert', 'Miller', 'robert.m@email.com', '+1-555-0106', 'hashed_password_6', TRUE),  
23 (2, 'Jennifer', 'Taylor', 'jennifer.t@email.com', '+1-555-0107', 'hashed_password_7', TRUE),  
24 (2, 'William', 'Anderson', 'william.a@email.com', '+1-555-0108', 'hashed_password_8', TRUE),  
25 (2, 'Linda', 'Thomas', 'linda.t@email.com', '+1-555-0109', 'hashed_password_9', TRUE);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Users	Properties	Bookings	Payments	Reviews	Messages
24	22	22	22	14	12

TEST QUERY & RESULT:

```
1 • SELECT UserID, FirstName, LastName, Email, RoleID FROM User LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows: |

	UserID	FirstName	LastName	Email	RoleID
▶	1	John	Smith	john.smith@email.com	1
	2	Emma	Johnson	emma.j@email.com	1
	3	Michael	Brown	michael.b@email.com	1
	4	Sarah	Davis	sarah.d@email.com	1
	5	David	Wilson	david.w@email.com	1

LOCATION TABLE

CREATE LOCATION TABLE STATEMENT:

```
36 CREATE TABLE Location (  
37     LocationID INT PRIMARY KEY AUTO_INCREMENT,  
38     Address VARCHAR(255) NOT NULL,  
39     City VARCHAR(100) NOT NULL,  
40     State VARCHAR(100),  
41     Country VARCHAR(100) NOT NULL,  
42     PostalCode VARCHAR(20),  
43     Latitude DECIMAL(10, 8),  
44     Longitude DECIMAL(11, 8),  
45     CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
46     INDEX idx_city (City),  
47     INDEX idx_country (Country)  
48 );  
49
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Total Tables Created
22

INSERT SAMPLE DATA:

```
1 INSERT INTO Location (Address, City, State, Country, PostalCode, Latitude, Longitude) VALUES  
2 ('123 Main St', 'New York', 'NY', 'USA', '10001', 40.712776, -74.005974),  
3 ('456 Park Ave', 'Los Angeles', 'CA', 'USA', '90001', 34.052235, -118.243683),  
4 ('789 Ocean Dr', 'Miami', 'FL', 'USA', '33139', 25.761681, -80.191788),  
5 ('101 Mountain Rd', 'Denver', 'CO', 'USA', '80202', 39.739235, -104.990250),  
6 ('202 River View', 'Chicago', 'IL', 'USA', '60601', 41.878113, -87.629799),  
7 ('303 Sunset Blvd', 'San Francisco', 'CA', 'USA', '94102', 37.774929, -122.419418),  
8 ('404 Lake St', 'Seattle', 'WA', 'USA', '98101', 47.606209, -122.332069),  
9 ('505 Forest Ave', 'Portland', 'OR', 'USA', '97201', 45.505106, -122.675026),  
10 ('606 Beach Rd', 'San Diego', 'CA', 'USA', '92101', 32.715736, -117.161087),  
11 ('707 Hill St', 'Austin', 'TX', 'USA', '73301', 30.267153, -97.743057),  
12 ('707 Hill St', 'Austin', 'TX', 'USA', '73301', 30.267153, -97.743057),  
13 ('808 Valley Rd', 'Phoenix', 'AZ', 'USA', '85001', 33.448376, -112.074036),  
14 ('909 Plaza Sq', 'Las Vegas', 'NV', 'USA', '89101', 36.169941, -115.139832);
```

TEST QUERY & RESULT:

```
1 SELECT LocationID, Address, City, Country FROM Location LIMIT 5;
```

LocationID	Address	City	Country
1	123 Main St	New York	USA
2	456 Park Ave	Los Angeles	USA
3	789 Ocean Dr	Miami	USA
4	101 Mountain Rd	Denver	USA
5	202 River View	Chicago	USA

PROPERTY TYPE TABLE

CREATE PROPERTYTYPE TABLE STATEMENT:

```
51 CREATE TABLE PropertyType (  
52     TypeID INT PRIMARY KEY AUTO_INCREMENT,  
53     TypeName VARCHAR(100) NOT NULL UNIQUE,  
54     Description TEXT,  
55     CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
56 );
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
Total Tables Created			
22			

INSERT SAMPLE DATA:

```
1 INSERT INTO PropertyType (TypeName, Description) VALUES  
2 ('Apartment', 'Self-contained housing unit in a building'),  
3 ('House', 'Detached residential building'),  
4 ('Villa', 'Luxury vacation home'),  
5 ('Condo', 'Individually owned unit in a complex'),  
6 ('Studio', 'Single room combining living, sleeping, and kitchen areas'),  
7 ('Loft', 'Open-plan living space, often converted from industrial use'),  
8 ('Cabin', 'Wooden house in a natural setting'),  
9 ('Castle', 'Historical fortified building'),  
10 ('Bungalow', 'Single-story house'),  
11 ('Treehouse', 'Structure built among trees'),  
12 ('Boat', 'Floating accommodation'),  
13 ('Farm', 'Agricultural property with accommodation');
```

TEST QUERY & RESULTS

```
1  
2 SELECT * FROM PropertyType LIMIT 5;
```

TypeID	TypeName	Description	CreatedAt
1	Apartment	Self-contained housing unit in a building	2026-03-08 16:53:36
2	House	Detached residential building	2026-03-08 16:53:36
3	Villa	Luxury vacation home	2026-03-08 16:53:36
4	Condo	Individually owned unit in a complex	2026-03-08 16:53:36
5	Studio	Single room combining living, sleeping, and kitch...	2026-03-08 16:53:36

PROPERTY TABLE

CREATE PROPERTY TABLE STATEMENT:

```
1 CREATE TABLE Property (  
2   PropertyID INT PRIMARY KEY AUTO_INCREMENT,  
3   HostID INT NOT NULL,  
4   TypeID INT NOT NULL,  
5   Title VARCHAR(200) NOT NULL,  
6   Description TEXT,  
7   LocationID INT NOT NULL,  
8   BasePrice DECIMAL(10, 2) NOT NULL CHECK (BasePrice > 0),  
9   CleaningFee DECIMAL(8, 2) DEFAULT 0,  
10  ServiceFee DECIMAL(8, 2) DEFAULT 0,  
11  MaxGuests INT NOT NULL CHECK (MaxGuests > 0),  
12  Bedrooms INT,  
13  Bathrooms DECIMAL(3, 1),  
14  SquareMeters INT,  
15  IsActive BOOLEAN DEFAULT TRUE,  
16  CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
17  UpdatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
18  FOREIGN KEY (HostID) REFERENCES User(UserID) ON DELETE CASCADE,  
19  FOREIGN KEY (TypeID) REFERENCES PropertyType(TypeID) ON DELETE RESTRICT,  
20  FOREIGN KEY (LocationID) REFERENCES Location(LocationID) ON DELETE RESTRICT,  
21  INDEX idx_host (HostID),  
22  INDEX idx_location (LocationID),  
23  INDEX idx_type (TypeID)  
24 );
```

INSERT SAMPLE DATA:

```
1 INSERT INTO Property (HostID, TypeID, Title, Description, LocationID, BasePrice, MaxGuests, Bedrooms, Bathrooms, SquareMeters) VALUES  
2 (6, 1, 'Cozy Downtown Apartment', 'Modern apartment in heart of NYC', 1, 120.00, 4, 2, 1, 75),  
3 (7, 2, 'Beachfront Villa', 'Luxury villa with private beach access', 3, 450.00, 8, 4, 3, 200),  
4 (8, 3, 'Mountain Retreat Cabin', 'Peaceful cabin with mountain views', 4, 180.00, 6, 3, 2, 120),  
5 (9, 1, 'City Center Loft', 'Industrial-style loft with high ceilings', 2, 200.00, 4, 1, 1, 90),  
6 (10, 4, 'Lakeside Condo', 'Beautiful condo overlooking the lake', 7, 150.00, 4, 2, 2, 85),  
7 (11, 5, 'Artistic Studio', 'Creative space for artists', 5, 90.00, 2, 1, 1, 50),  
8 (12, 6, 'Urban Loft', 'Modern loft in trendy neighborhood', 6, 220.00, 3, 1, 1, 70),  
9 (18, 7, 'Forest Cabin', 'Secluded cabin in the woods', 8, 130.00, 5, 2, 1, 80),  
10 (19, 8, 'Historic Castle', '15th century castle with modern amenities', 16, 800.00, 12, 6, 5, 500),  
11 (20, 9, 'Beach Bungalow', 'Traditional bungalow steps from the beach', 3, 160.00, 4, 2, 1, 70),  
12 (21, 10, 'Treehouse Escape', 'Unique treehouse experience', 8, 190.00, 3, 1, 1, 40),  
13 (6, 11, 'Luxury Yacht', 'Private yacht for overnight stays', 3, 650.00, 6, 3, 2, NULL),  
14 (7, 12, 'Vintage Airstream', 'Retro camper with modern comforts', 4, 110.00, 3, 1, 1, 25),  
15 (8, 13, 'Farm Stay', 'Working farm with guest cottage', 4, 140.00, 5, 2, 1, 90),  
16 (9, 1, 'Penthouse with View', 'Top-floor apartment with panoramic views', 1, 350.00, 6, 3, 2, 120),  
17 (10, 2, 'Family Home', 'Spacious house perfect for families', 2, 280.00, 8, 4, 3, 180);
```

Output

#	Time	Action	Message
37	16:53:36	SELECT CONCAT(COUNT(*), 'property types inserted') AS Status FROM PropertyType LIMIT...	1 row(s) returned
38	16:53:36	INSERT INTO Property (HostID, TypeID, Title, Description, LocationID, BasePrice, MaxGuests, Bedrooms, Bathrooms, SquareMeters) VALUES	22 row(s) affected Records: 22 Duplicates: 0 Warnings: 0

TEST QUERY & RESULT:

```
1 SELECT PropertyID, Title, BasePrice, MaxGuests FROM Property LIMIT 5;
```

PropertyID	Title	BasePrice	MaxGuests
1	Cozy Downtown Apartment	120.00	4
2	Beachfront Villa	450.00	8
3	Mountain Retreat Cabin	180.00	6
4	City Center Loft	200.00	4
5	Lakeside Condo	150.00	4

AMENITY TABLE

CREATE AMENITY TABLE:

```
85 • CREATE TABLE Amenity (  
86     AmenityID INT PRIMARY KEY AUTO_INCREMENT,  
87     AmenityName VARCHAR(100) NOT NULL UNIQUE,  
88     Category VARCHAR(50),  
89     IconClass VARCHAR(50),  
90     CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
91 );
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Total Tables Created
22

INSERT SAMPLE DATA:

```
1 • INSERT INTO Amenity (AmenityName, Category) VALUES  
2 ('WiFi', 'Essentials'),  
3 ('Kitchen', 'Essentials'),  
4 ('Washer', 'Essentials'),  
5 ('Dryer', 'Essentials'),  
6 ('Air Conditioning', 'Essentials'),  
7 ('Heating', 'Essentials'),  
8 ('TV', 'Entertainment'),  
9 ('Cable TV', 'Entertainment'),  
10 ('Netflix', 'Entertainment'),  
11 ('Pool', 'Outdoor'),  
12 ('Hot Tub', 'Outdoor'),  
13 ('Patio', 'Outdoor'),  
14 ('BBQ Grill', 'Outdoor'),  
15 ('Garden', 'Outdoor'),  
16 ('Parking', 'Logistics');
```

TEST QUERY & RESULT

1 • SELECT * FROM Amenity LIMIT 5;

	AmenityID	AmenityName	Category	IconClass	CreatedAt
▶	1	WiFi	Essentials	NULL	2026-03-08 16:53:36
	2	Kitchen	Essentials	NULL	2026-03-08 16:53:36
	3	Washer	Essentials	NULL	2026-03-08 16:53:36
	4	Dryer	Essentials	NULL	2026-03-08 16:53:36
	5	Air Conditioning	Essentials	NULL	2026-03-08 16:53:36

PROPERTY AMENITY TABLE

CREATE PROPERTY AMENITY STATEMENT

```
94 CREATE TABLE PropertyAmenity (  
95     PropertyID INT NOT NULL,  
96     AmenityID INT NOT NULL,  
97     AddedDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
98     PRIMARY KEY (PropertyID, AmenityID),  
99     FOREIGN KEY (PropertyID) REFERENCES Property(PropertyID) ON DELETE CASCADE,  
100    FOREIGN KEY (AmenityID) REFERENCES Amenity(AmenityID) ON DELETE CASCADE,  
101    INDEX idx_property (PropertyID),  
102    INDEX idx_amenity (AmenityID)  
103 );  
104
```

INSERT SAMPLE DATA

```
1 INSERT INTO PropertyAmenity (PropertyID, AmenityID) VALUES  
2 (1, 1), (1, 2), (1, 5), (1, 7), (1, 15),  
3 (2, 1), (2, 2), (2, 5), (2, 10), (2, 11), (2, 13), (2, 21),  
4 (3, 1), (3, 2), (3, 6), (3, 19), (3, 22),  
5 (4, 1), (4, 2), (4, 5), (4, 7), (4, 9), (4, 20),  
6 (5, 1), (5, 2), (5, 3), (5, 4), (5, 5), (5, 10), (5, 12),  
7 (6, 1), (6, 2), (6, 5), (6, 7),  
8 (7, 1), (7, 2), (7, 5), (7, 7), (7, 9), (7, 17),  
9 (8, 1), (8, 2), (8, 6), (8, 19), (8, 22),  
10 (10, 1), (10, 2), (10, 5), (10, 12), (10, 13), (10, 21);
```

TEST QUERY & RESULT

```
1 SELECT * FROM PropertyAmenity LIMIT 10;
```

PropertyID	AmenityID	AddedDate
1	1	2026-03-10 06:24:55
1	2	2026-03-10 06:24:55
1	5	2026-03-10 06:24:55
1	7	2026-03-10 06:24:55
1	15	2026-03-10 06:24:55
2	1	2026-03-10 06:24:55
2	2	2026-03-10 06:24:55
2	5	2026-03-10 06:24:55
2	10	2026-03-10 06:24:55
2	11	2026-03-10 06:24:55

BOOKING STATUS TABLE

CREATE BOOKINGSTATUS TABLE

```
105 -- 8. BOOKINGSTATUS TABLE
106 • CREATE TABLE BookingStatus (
107     StatusID INT PRIMARY KEY AUTO_INCREMENT,
108     StatusName VARCHAR(50) NOT NULL UNIQUE,
109     Description VARCHAR(255),
110     IsActive BOOLEAN DEFAULT TRUE,
111     CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
112 );
113
```

INSERT SAMPLE DATA

```
168 • INSERT INTO BookingStatus (StatusName, Description) VALUES
169     ('Pending', 'Booking request sent, awaiting host confirmation'),
170     ('Confirmed', 'Booking confirmed by host'),
171     ('Cancelled', 'Booking cancelled by guest or host'),
172     ('Completed', 'Stay has been completed'),
173     ('No-show', 'Guest did not arrive'),
174     ('Refunded', 'Booking cancelled and refund processed');
175
```

TEST QUERY & RESULT

```
1 • SELECT * FROM BookingStatus;
```

Result Grid Filter Rows: Export: Wrap Cell Content:					
	StatusID	StatusName	Description	IsActive	CreatedAt
▶	1	Pending	Booking request sent, awaiting host confirmation	1	2026-03-10 06:24:55
	2	Confirmed	Booking confirmed by host	1	2026-03-10 06:24:55
	3	Cancelled	Booking cancelled by guest or host	1	2026-03-10 06:24:55
	4	Completed	Stay has been completed	1	2026-03-10 06:24:55
	5	No-show	Guest did not arrive	1	2026-03-10 06:24:55
	6	Refunded	Booking cancelled and refund processed	1	2026-03-10 06:24:55

BOOKING TABLE

CREATE BOOKING TABLE STATEMENT

```
1 CREATE TABLE Booking (  
2     BookingID INT PRIMARY KEY AUTO_INCREMENT,  
3     GuestID INT NOT NULL,  
4     PropertyID INT NOT NULL,  
5     CheckInDate DATE NOT NULL,  
6     CheckOutDate DATE NOT NULL,  
7     Nights INT GENERATED ALWAYS AS (DATEDIFF(CheckOutDate, CheckInDate)) STORED,  
8     GuestsCount INT NOT NULL DEFAULT 1 CHECK (GuestsCount > 0),  
9     BasePrice DECIMAL(10, 2) NOT NULL,  
10    CleaningFee DECIMAL(8, 2) DEFAULT 0,  
11    ServiceFee DECIMAL(8, 2) DEFAULT 0,  
12    TotalPrice DECIMAL(12, 2) NOT NULL,  
<
```

INSERT SAMPLE DATA:

```
1 INSERT INTO Booking (GuestID, PropertyID, CheckInDate, CheckOutDate, GuestsCount, BasePrice, TotalPrice, StatusID) VALUES  
2 (1, 1, '2024-03-15', '2024-03-20', 2, 120.00, 600.00, 2),  
3 (2, 2, '2024-04-10', '2024-04-17', 4, 450.00, 3150.00, 2),  
4 (3, 3, '2024-05-01', '2024-05-07', 3, 180.00, 1080.00, 2),  
5 (22, 10, '2024-07-15', '2024-07-22', 3, 160.00, 1120.00, 2),  
6 (23, 11, '2024-08-01', '2024-08-07', 2, 190.00, 1140.00, 2),  
7 (24, 12, '2024-09-10', '2024-09-15', 4, 650.00, 3250.00, 2),  
8 (1, 13, '2024-10-05', '2024-10-10', 2, 110.00, 550.00, 2),  
9 (2, 14, '2024-11-15', '2024-11-20', 3, 140.00, 700.00, 2),  
10 (3, 15, '2024-12-01', '2024-12-08', 4, 350.00, 2450.00, 2),  
11 (4, 16, '2024-03-10', '2024-03-15', 5, 280.00, 1400.00, 2),  
12 (14, 18, '2024-05-15', '2024-05-20', 3, 210.00, 1050.00, 2);
```

TEST QUERY & RESULT

```
1 • SELECT BookingID, GuestID, PropertyID, CheckInDate, CheckOutDate, TotalPrice FROM Booking LIMIT 5;
```

	BookingID	GuestID	PropertyID	CheckInDate	CheckOutDate	TotalPrice
▶	1	1	1	2024-03-15	2024-03-20	600.00
	2	2	2	2024-04-10	2024-04-17	3150.00
	3	3	3	2024-05-01	2024-05-07	1080.00
	4	4	4	2024-06-15	2024-06-20	1000.00
	5	5	5	2024-07-01	2024-07-10	1350.00

PAYMENT METHOD TABLE

CREATE PAYMENT METHOD TABLE

```
142 -- 10. PAYMENTMETHOD TABLE
143 CREATE TABLE PaymentMethod (
144     MethodID INT PRIMARY KEY AUTO_INCREMENT,
145     MethodName VARCHAR(50) NOT NULL UNIQUE,
146     Processor VARCHAR(50),
147     IsActive BOOLEAN DEFAULT TRUE,
148     CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
149 );
```

INSERT SAMPLE DATA

```
209 INSERT INTO PaymentMethod (MethodName, Processor) VALUES
210     ('Credit Card', 'Stripe'),
211     ('PayPal', 'PayPal'),
212     ('Apple Pay', 'Apple'),
213     ('Google Pay', 'Google'),
214     ('Bank Transfer', 'Direct'),
215     ('Cash', 'Manual');
```

TEST QUERY & RESULT

```
1 SELECT * FROM PaymentMethod;
```

	MethodID	MethodName	Processor	IsActive	CreatedAt
▶	1	Credit Card	Stripe	1	2026-03-14 20:36:05
	2	PayPal	PayPal	1	2026-03-14 20:36:05
	3	Apple Pay	Apple	1	2026-03-14 20:36:05
	4	Google Pay	Google	1	2026-03-14 20:36:05
	5	Bank Transfer	Direct	1	2026-03-14 20:36:05
	6	Cash	Manual	1	2026-03-14 20:36:05

PAYMENT STATUS TABLE

CREATE PAYMENTSTATUS STATEMENT:

```
151  -- 11. PAYMENTSTATUS TABLE
152  CREATE TABLE PaymentStatus (
153      PaymentStatusID INT PRIMARY KEY AUTO_INCREMENT,
154      StatusName VARCHAR(50) NOT NULL UNIQUE,
155      Description VARCHAR(255),
156      CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
157  );
158
```

INSERT SAMPLE DATA:

```
219  -- 11. Insert Payment Statuses (no dependencies)
220  INSERT INTO PaymentStatus (StatusName) VALUES
221      ('Pending'),
222      ('Completed'),
223      ('Failed'),
224      ('Refunded'),
225      ('Partially Refunded');
```

TEST QUERY & RESULT:

```
1  SELECT * FROM PaymentStatus;
```

	PaymentStatusID	StatusName	Description	CreatedAt
▶	1	Pending	NULL	2026-03-15 11:53:47
	2	Completed	NULL	2026-03-15 11:53:47
	3	Failed	NULL	2026-03-15 11:53:47
	4	Refunded	NULL	2026-03-15 11:53:47
	5	Partially Refunded	NULL	2026-03-15 11:53:47
✱	NULL	NULL	NULL	NULL

PAYMENT TABLE

CREATE PAYMENT TABLE STATEMENT:

```
1 CREATE TABLE Payment (  
2     PaymentID INT PRIMARY KEY AUTO_INCREMENT,  
3     BookingID INT NOT NULL,  
4     Amount DECIMAL(10, 2) NOT NULL CHECK (Amount > 0),  
5     PaymentDate DATETIME NOT NULL,  
6     MethodID INT NOT NULL,  
7     StatusID INT NOT NULL,  
8     TransactionID VARCHAR(100) UNIQUE,  
9     FOREIGN KEY (BookingID) REFERENCES Booking(BookingID) ON DELETE CASCADE,  
10    FOREIGN KEY (MethodID) REFERENCES PaymentMethod(MethodID) ON DELETE RESTRICT,  
11    FOREIGN KEY (StatusID) REFERENCES PaymentStatus(PaymentStatusID) ON DELETE RESTRICT  
12 );
```

INSERT SAMPLE DATA:

```
1 INSERT INTO Payment (BookingID, Amount, PaymentDate, MethodID, StatusID, TransactionID) VALUES  
2 (1, 600.00, '2024-03-01 10:30:00', 1, 2, 'TXN001'),  
3 (2, 3150.00, '2024-03-25 14:45:00', 2, 2, 'TXN002'),  
4 (3, 1080.00, '2024-04-15 09:15:00', 1, 2, 'TXN003'),  
5 (4, 1000.00, '2024-05-20 16:30:00', 3, 2, 'TXN004'),  
6 (5, 1350.00, '2024-06-01 11:00:00', 1, 2, 'TXN005'),  
7 (6, 450.00, '2024-03-10 13:20:00', 2, 2, 'TXN006'),  
8 (7, 1540.00, '2024-03-28 15:45:00', 1, 2, 'TXN007'),  
9 (8, 650.00, '2024-05-05 10:10:00', 1, 2, 'TXN008'),  
10 (9, 5600.00, '2024-05-20 14:30:00', 2, 2, 'TXN009'),  
11 (10, 1120.00, '2024-06-25 12:15:00', 1, 2, 'TXN010'),  
12 (11, 1140.00, '2024-07-15 09:45:00', 3, 2, 'TXN011'),  
13 (12, 3250.00, '2024-08-20 16:00:00', 1, 2, 'TXN012'),  
14 (13, 550.00, '2024-09-10 11:30:00', 2, 2, 'TXN013'),  
15 (14, 700.00, '2024-10-05 14:20:00', 1, 2, 'TXN014'),  
16 (18, 1050.00, '2024-04-28 12:10:00', 3, 2, 'TXN018'),  
17 (19, 900.00, '2024-05-25 09:30:00', 1, 2, 'TXN019'),  
18 (20, 1680.00, '2024-07-10 16:45:00', 1, 2, 'TXN020'),  
19 (21, 1190.00, '2024-07-30 14:00:00', 2, 2, 'TXN021'),  
20 (22, 5250.00, '2024-08-20 11:15:00', 1, 2, 'TXN022');
```

INSERT SAMPLE DATA:

1 • SELECT PaymentID, BookingID, Amount, MethodID, StatusID FROM Payment LIMIT 5;

PaymentID	BookingID	Amount	MethodID	StatusID
1	1	600.00	1	2
2	2	3150.00	2	2
3	3	1080.00	1	2
4	4	1000.00	3	2
5	5	1350.00	1	2

PAYOUT TABLE

CREATE PAYOUT TABLE STATEMENT:

```
1 CREATE TABLE Payout (  
2     PayoutID INT PRIMARY KEY AUTO_INCREMENT,  
3     HostID INT NOT NULL,  
4     BookingID INT NOT NULL,  
5     Amount DECIMAL(10, 2) NOT NULL CHECK (Amount > 0),  
6     PayoutDate DATE NOT NULL,  
7     Status VARCHAR(50) DEFAULT 'Pending',  
8     TransactionRef VARCHAR(100),  
9     CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
10    FOREIGN KEY (HostID) REFERENCES User(UserID) ON DELETE CASCADE,  
11    FOREIGN KEY (BookingID) REFERENCES Booking(BookingID) ON DELETE CASCADE  
12 );
```

INSERT SAMPLE DATA

```
1 INSERT INTO Payout (HostID, BookingID, Amount, PayoutDate, Status) VALUES  
2 (6, 1, 540.00, '2024-03-25', 'Completed'),  
3 (7, 2, 2835.00, '2024-04-20', 'Completed'),  
4 (8, 3, 972.00, '2024-05-10', 'Completed'),  
5 (9, 4, 900.00, '2024-06-25', 'Completed'),  
6 (10, 5, 1215.00, '2024-07-15', 'Completed'),  
7 (11, 6, 405.00, '2024-04-05', 'Completed'),  
8 (12, 7, 1386.00, '2024-04-15', 'Completed'),  
9 (18, 8, 585.00, '2024-05-30', 'Completed'),  
10 (19, 9, 5040.00, '2024-06-12', 'Completed'),  
11 (20, 10, 1008.00, '2024-07-30', 'Completed'),  
12 (21, 11, 1026.00, '2024-08-10', 'Completed');
```

TEST QUERY & RESULT

```
1 SELECT PayoutID, HostID, BookingID, Amount, Status FROM Payout LIMIT 5;
```

Result Grid					
Filter Rows:					
	PayoutID	HostID	BookingID	Amount	Status
1	1	6	1	540.00	Completed
2	2	7	2	2835.00	Completed
3	3	8	3	972.00	Completed
4	4	9	4	900.00	Completed
5	5	10	5	1215.00	Completed

REVIEW TABLE(Recursive Relationship)

CREATE REVIEW TABLE STATEMENT

```
1 CREATE TABLE Review (  
2     ReviewID INT PRIMARY KEY AUTO_INCREMENT,  
3     BookingID INT NOT NULL UNIQUE,  
4     ReviewerID INT NOT NULL,  
5     RevieweeID INT NOT NULL,  
6     Rating INT NOT NULL CHECK (Rating >= 1 AND Rating <= 5),  
7     Comment TEXT,  
8     Response TEXT,  
9     ReviewDate DATE DEFAULT (CURRENT_DATE),  
10    FOREIGN KEY (BookingID) REFERENCES Booking(BookingID) ON DELETE CASCADE,  
11    FOREIGN KEY (ReviewerID) REFERENCES User(UserID) ON DELETE CASCADE,  
12    FOREIGN KEY (RevieweeID) REFERENCES User(UserID) ON DELETE CASCADE,  
13    CONSTRAINT chk_not_self_review CHECK (ReviewerID != RevieweeID));
```

INERT SAMPLE DATA

```
1 INSERT INTO Review (BookingID, ReviewerID, RevieweeID, Rating, Comment, Response) VALUES  
2 (1, 1, 6, 5, 'Great apartment, very clean and well located!', 'Thank you, John! You were a perfect guest.'),  
3 (2, 2, 7, 4, 'Beautiful villa, but WiFi was slow.', 'Thanks Emma! We will upgrade the WiFi.'),  
4 (3, 3, 8, 5, 'Perfect mountain getaway!', 'So glad you enjoyed it, Michael!'),  
5 (4, 4, 9, 4, 'Cool loft, amazing design.', 'Thank you Sarah!'),  
6 (5, 5, 10, 5, 'Best lake view ever!', 'Come back anytime, David!'),  
7 (6, 14, 11, 3, 'Studio was smaller than expected.', 'Thanks for your feedback, Lisa.'),  
8 (7, 15, 12, 5, 'Amazing urban experience!', 'Welcome back anytime, Daniel!'),  
9 (8, 16, 18, 4, 'Peaceful forest retreat.', 'Thank you Nancy!'),  
10 (9, 17, 19, 5, 'Once in a lifetime experience!', 'We are happy you enjoyed the castle!'),  
11 (10, 22, 20, 5, 'Perfect beach bungalow!', 'Thanks Mark!'),  
12 (11, 23, 21, 4, 'Fun treehouse, kids loved it!', 'Glad your family enjoyed it!'),  
13 (12, 6, 1, 5, 'Excellent guest, very respectful.', 'Thank you Robert!');
```

TEST QUERY & RESULT

```
1 SELECT ReviewID, BookingID, ReviewerID, RevieweeID, Rating FROM Review LIMIT 5;
```

ReviewID	BookingID	ReviewerID	RevieweeID	Rating
1	1	1	6	5
2	2	2	7	4
3	3	3	8	5
4	4	4	9	4
5	5	5	10	5

AVAILABILITY CALENDAR TABLE

CREATE AVAILABILITY CALENDAR STATEMENT:

```
218 CREATE TABLE AvailabilityCalendar (  
219     CalendarID INT PRIMARY KEY AUTO_INCREMENT,  
220     PropertyID INT NOT NULL,  
221     CalendarDate DATE NOT NULL,  
222     IsAvailable BOOLEAN DEFAULT TRUE,  
223     CustomPrice DECIMAL(10, 2),  
224     CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
225     FOREIGN KEY (PropertyID) REFERENCES Property(PropertyID) ON DELETE CASCADE,  
226     UNIQUE KEY uk_property_date (PropertyID, CalendarDate),  
227     INDEX idx_property (PropertyID),  
228     INDEX idx_date (CalendarDate)  
229 );
```

INSERT SAMPLE DATA:

```
1 INSERT INTO AvailabilityCalendar (PropertyID, CalendarDate, IsAvailable, CustomPrice) VALUES  
2 (1, '2024-03-15', FALSE, NULL),  
3 (1, '2024-03-16', FALSE, NULL),  
4 (1, '2024-03-17', FALSE, NULL),  
5 (1, '2024-03-18', FALSE, NULL),  
6 (1, '2024-03-19', FALSE, NULL),  
7 (1, '2024-03-20', FALSE, NULL),  
8 (1, '2024-03-21', TRUE, NULL),  
9 (1, '2024-03-22', TRUE, NULL),  
10 (2, '2024-04-10', FALSE, NULL),  
11 (2, '2024-04-11', FALSE, NULL),  
12 (2, '2024-04-12', FALSE, NULL),  
13 (2, '2024-04-13', FALSE, NULL),  
14 (2, '2024-04-14', FALSE, NULL),  
15 (2, '2024-04-15', FALSE, NULL),  
16 (2, '2024-04-16', FALSE, NULL),  
17 (2, '2024-04-17', FALSE, NULL),  
18 (2, '2024-04-18', TRUE, NULL),  
19 (3, '2024-05-01', FALSE, NULL),  
20 (3, '2024-05-02', FALSE, NULL);
```

TEST QUERY & RESULT:

```
1 SELECT * FROM AvailabilityCalendar LIMIT 5;
```

CalendarID	PropertyID	CalendarDate	IsAvailable	CustomPrice	CreatedAt
1	1	2024-03-15	0	NULL	2026-03-15 11:53:47
2	1	2024-03-16	0	NULL	2026-03-15 11:53:47
3	1	2024-03-17	0	NULL	2026-03-15 11:53:47
4	1	2024-03-18	0	NULL	2026-03-15 11:53:47
5	1	2024-03-19	0	NULL	2026-03-15 11:53:47

MESSAGE TABLE

CREATE MESSAGE TABLE STATEMENT

```
1 CREATE TABLE Message (  
2     MessageID INT PRIMARY KEY AUTO_INCREMENT,  
3     SenderID INT NOT NULL,  
4     ReceiverID INT NOT NULL,  
5     BookingID INT,  
6     MessageText TEXT NOT NULL,  
7     SentDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
8     ReadDate TIMESTAMP NULL,  
9     IsRead BOOLEAN DEFAULT FALSE,  
10    FOREIGN KEY (SenderID) REFERENCES User(UserID) ON DELETE CASCADE,  
11    FOREIGN KEY (ReceiverID) REFERENCES User(UserID) ON DELETE CASCADE,  
12    FOREIGN KEY (BookingID) REFERENCES Booking(BookingID) ON DELETE SET NULL  
13 );
```

INSERT SAMPLE DATA

```
337 INSERT INTO Message (SenderID, ReceiverID, BookingID, MessageText) VALUES  
338 (1, 6, 1, 'Hi, is early check-in possible?'),  
339 (6, 1, 1, 'Yes, 1 PM check-in is fine.'),  
340 (2, 7, 2, 'Do you provide beach towels?'),  
341 (7, 2, 2, 'Yes, beach towels are provided.'),  
342 (3, 8, 3, 'Is the road to the cabin accessible in winter?'),  
343 (8, 3, 3, 'Yes, the road is plowed regularly.'),  
344 (4, 9, 4, 'What is the parking situation?'),  
345 (9, 4, 4, 'Street parking available with permit.'),  
346 (5, 10, 5, 'Is fishing allowed from the dock?'),  
347 (10, 5, 5, 'Yes, fishing is allowed.'),  
348 (1, 6, NULL, 'Thinking of booking again next month!'),  
349 (6, 1, NULL, 'Great! Let me know dates.');
```

TEST QUERY & RESULT

```
1 SELECT MessageID, SenderID, ReceiverID, LEFT(MessageText, 30) AS Message FROM Message LIMIT 5;
```

MessageID	SenderID	ReceiverID	Message
1	1	6	Hi, is early check-in possible
2	6	1	Yes, 1 PM check-in is fine.
3	2	7	Do you provide beach towels?
4	7	2	Yes, beach towels are provided
5	3	8	Is the road to the cabin acces

DISPUTE TABLE

CREATE DISPUTE TABLE STATEMENT

```
1 CREATE TABLE Dispute (  
2     DisputeID INT PRIMARY KEY AUTO_INCREMENT,  
3     BookingID INT NOT NULL,  
4     RaisedByID INT NOT NULL,  
5     Title VARCHAR(200) NOT NULL,  
6     Description TEXT NOT NULL,  
7     Status VARCHAR(50) DEFAULT 'Open',  
8     Resolution TEXT,  
9     CreatedDate DATE DEFAULT (CURRENT_DATE),  
10    ResolvedDate DATE,  
11    FOREIGN KEY (BookingID) REFERENCES Booking(BookingID) ON DELETE CASCADE,  
12    FOREIGN KEY (RaisedByID) REFERENCES User(UserID) ON DELETE CASCADE  
13 );
```

INSERT SAMPLE DATA:

```
354 INSERT INTO Dispute (BookingID, RaisedByID, Title, Description, Status) VALUES  
355 (6, 14, 'Cleanliness Issue', 'The studio was not properly cleaned before arrival.', 'Resolved'),  
356 (7, 15, 'Noise Complaint', 'Construction noise during stay not mentioned.', 'Open'),  
357 (11, 23, 'Safety Concern', 'Treehouse ladder needs repair.', 'In Progress');
```

TEST QUERY & RESULT:

```
1 SELECT * FROM Dispute;
```

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content:								
	DisputeID	BookingID	RaisedByID	Title	Description	Status	Resolution	CreatedDate
1	1	6	14	Cleanliness Issue	The studio was not properly cleaned before arrival.	Resolved	NULL	2026-03-15
2	2	7	15	Noise Complaint	Construction noise during stay not mentioned.	Open	NULL	2026-03-15
3	3	11	23	Safety Concern	Treehouse ladder needs repair.	In Progress	NULL	2026-03-15

AUDITLOG TABLE

CREATE AUDITLOG TABLE STATEMENT:

INSERT SAMPLE DATA:

```
1 CREATE TABLE AuditLog (  
2     LogID INT PRIMARY KEY AUTO_INCREMENT,  
3     UserID INT,  
4     Action VARCHAR(255) NOT NULL,  
5     EntityType VARCHAR(50),  
6     EntityID INT,  
7     Details JSON,  
8     IPAddress VARCHAR(45),  
9     Timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
10    FOREIGN KEY (UserID) REFERENCES User(UserID) ON DELETE SET NULL  
11 );
```

```
362 INSERT INTO AuditLog (UserID, Action, EntityType, EntityID, Details) VALUES  
363 (1, 'BOOKING_CREATED', 'Booking', 1, '{"amount": 600, "property": "Cozy Downtown Apartment"}'),  
364 (2, 'BOOKING_CREATED', 'Booking', 2, '{"amount": 3150, "property": "Beachfront Villa"}'),  
365 (6, 'PROPERTY_UPDATED', 'Property', 1, '{"field": "price", "old": 110, "new": 120}'),  
366 (13, 'USER_VERIFIED', 'User', 3, '{"verified": true}'),  
367 (13, 'DISPUTE_REVIEWED', 'Dispute', 1, '{"resolution": "refund_issued", "amount": 50}');  
368
```

TEST QUERY & RESULT:

```
1 SELECT LogID, UserID, Action, EntityType FROM AuditLog;
```

Result Grid				
Filter Rows:				
	LogID	UserID	Action	EntityType
1	1	1	BOOKING_CREATED	Booking
2	2	2	BOOKING_CREATED	Booking
3	6	6	PROPERTY_UPDATED	Property
4	13	13	USER_VERIFIED	User
5	13	13	DISPUTE_REVIEWED	Dispute

SOCIAL LINK TABLE

CREATE SOCIAL LINK STATEMENT:

```
285 • CREATE TABLE SocialLink (  
286     LinkID INT PRIMARY KEY AUTO_INCREMENT,  
287     UserID INT NOT NULL,  
288     Platform VARCHAR(50) NOT NULL,  
289     ProfileURL VARCHAR(255) NOT NULL,  
290     ConnectedDate DATE DEFAULT (CURRENT_DATE),  
291     IsVerified BOOLEAN DEFAULT FALSE,  
292     CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
293     FOREIGN KEY (UserID) REFERENCES User(UserID) ON DELETE CASCADE,  
294     INDEX idx_user (UserID),  
295     INDEX idx_platform (Platform)  
296 );
```

INSERT SAMPLE DATA:

```
372 • INSERT INTO SocialLink (UserID, Platform, ProfileURL, IsVerified) VALUES  
373     (1, 'Facebook', 'https://facebook.com/johnsmith', TRUE),  
374     (2, 'LinkedIn', 'https://linkedin.com/in/emmajohnson', TRUE),  
375     (6, 'Facebook', 'https://facebook.com/robertmiller', TRUE),  
376     (7, 'Instagram', 'https://instagram.com/jennifert', TRUE),  
377     (13, 'Twitter', 'https://twitter.com/airbnbadmin', TRUE);
```

TEST QUERY & RESULT:

```
1 • SELECT * FROM SocialLink;
```

LinkID	UserID	Platform	ProfileURL	ConnectedDate	IsVerified	CreatedAt
1	1	Facebook	https://facebook.com/johnsmith	2026-03-19	1	2026-03-19 06:42:38
2	2	LinkedIn	https://linkedin.com/in/emmajohnson	2026-03-19	1	2026-03-19 06:42:38
3	6	Facebook	https://facebook.com/robertmiller	2026-03-19	1	2026-03-19 06:42:38
4	7	Instagram	https://instagram.com/jennifert	2026-03-19	1	2026-03-19 06:42:38
5	13	Twitter	https://twitter.com/airbnbadmin	2026-03-19	1	2026-03-19 06:42:38

HOST REPORT TABLE

CREATE HOSTREPORT TABLE STATEMENT:

```
1 CREATE TABLE HostReport (  
2     ReportID INT PRIMARY KEY AUTO_INCREMENT,  
3     HostID INT NOT NULL,  
4     PeriodStart DATE NOT NULL,  
5     PeriodEnd DATE NOT NULL,  
6     TotalBookings INT DEFAULT 0,  
7     TotalNights INT DEFAULT 0,  
8     TotalRevenue DECIMAL(12, 2) DEFAULT 0,  
9     AvgRating DECIMAL(3, 2),  
10    CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
11    FOREIGN KEY (HostID) REFERENCES User(UserID) ON DELETE CASCADE  
12 );
```

INSERT SAMPLE DATA:

```
382 INSERT INTO HostReport (HostID, PeriodStart, PeriodEnd, TotalBookings, TotalNights, TotalRevenue, AvgRating) VALUES  
383 (6, '2024-01-01', '2024-03-31', 3, 21, 4500.00, 4.8),  
384 (7, '2024-01-01', '2024-03-31', 2, 14, 3700.00, 4.5),  
385 (8, '2024-01-01', '2024-03-31', 2, 13, 2050.00, 4.7),  
386 (9, '2024-01-01', '2024-03-31', 2, 10, 1900.00, 4.4),  
387 (10, '2024-01-01', '2024-03-31', 2, 15, 2565.00, 5.0);  
...
```

TEST QUERY & RESULT:

```
1 SELECT * FROM HostReport;
```

ReportID	HostID	PeriodStart	PeriodEnd	TotalBookings	TotalNights	TotalRevenue	AvgRating	CreatedAt
1	6	2024-01-01	2024-03-31	3	21	4500.00	4.80	2026-03-19 06:42:38
2	7	2024-01-01	2024-03-31	2	14	3700.00	4.50	2026-03-19 06:42:38
3	8	2024-01-01	2024-03-31	2	13	2050.00	4.70	2026-03-19 06:42:38
4	9	2024-01-01	2024-03-31	2	10	1900.00	4.40	2026-03-19 06:42:38
5	10	2024-01-01	2024-03-31	2	15	2565.00	5.00	2026-03-19 06:42:38

CANCELLATION POLICY TABLE

CREATE CANCELLATIONPOLICY TABLE STATEMENT:

```
315 • CREATE TABLE CancellationPolicy (  
316     PolicyID INT PRIMARY KEY AUTO_INCREMENT,  
317     PropertyID INT NOT NULL UNIQUE,  
318     PolicyType VARCHAR(50) NOT NULL,  
319     RefundPercentage INT CHECK (RefundPercentage >= 0 AND RefundPercentage <= 100),  
320     DaysBeforeCheckin INT,  
321     Description TEXT,  
322     CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
323     FOREIGN KEY (PropertyID) REFERENCES Property(PropertyID) ON DELETE CASCADE,  
324     INDEX idx_property (PropertyID)  
325 );
```

INSERT SAMPLE DATA:

```
392 • INSERT INTO CancellationPolicy (PropertyID, PolicyType, RefundPercentage, DaysBeforeCheckin, Description) VALUES  
393 (1, 'Flexible', 100, 2, 'Full refund if cancelled 2 days before check-in'),  
394 (2, 'Moderate', 50, 7, '50% refund if cancelled 7 days before check-in'),  
395 (3, 'Strict', 0, 14, 'No refund if cancelled within 14 days'),  
396 (4, 'Flexible', 100, 1, 'Full refund 1 day before'),  
397 (5, 'Moderate', 50, 5, '50% refund 5 days before');  
398
```

TEST QUERY & RESULT:

```
1 • SELECT * FROM CancellationPolicy;
```

Result Grid Filter Rows: Export: Wrap Cell Content:							
	PolicyID	PropertyID	PolicyType	RefundPercentage	DaysBeforeCheckin	Description	CreatedAt
▶	1	1	Flexible	100	2	Full refund if cancelled 2 days before check-in	2026-03-19 06:42:38
	2	2	Moderate	50	7	50% refund if cancelled 7 days before check-in	2026-03-19 06:42:38
	3	3	Strict	0	14	No refund if cancelled within 14 days	2026-03-19 06:42:38
	4	4	Flexible	100	1	Full refund 1 day before	2026-03-19 06:42:38
	5	5	Moderate	50	5	50% refund 5 days before	2026-03-19 06:42:38

VERIFICATION TABLE

CREATE VERIFICATION TABLE STATEMENT:

```
1 CREATE TABLE Verification (  
2     VerificationID INT PRIMARY KEY AUTO_INCREMENT,  
3     UserID INT NOT NULL,  
4     DocumentType VARCHAR(50) NOT NULL,  
5     DocumentURL VARCHAR(255) NOT NULL,  
6     Status VARCHAR(50) DEFAULT 'Pending',  
7     VerifiedBy INT,  
8     VerifiedDate DATE,  
9     FOREIGN KEY (UserID) REFERENCES User(UserID),  
10    FOREIGN KEY (VerifiedBy) REFERENCES User(UserID) ON DELETE SET NULL  
11 );
```

INSERT SAMPLE DATA:

```
402 INSERT INTO Verification (UserID, DocumentType, DocumentURL, Status, VerifiedDate) VALUES  
403 (1, 'Passport', 'https://storage.example.com/passport1.jpg', 'Verified', '2024-01-15'),  
404 (2, 'Drivers License', 'https://storage.example.com/dl2.jpg', 'Verified', '2024-01-20'),  
405 (6, 'Passport', 'https://storage.example.com/passport6.jpg', 'Verified', '2024-02-01'),  
406 (7, 'ID Card', 'https://storage.example.com/id7.jpg', 'Verified', '2024-02-05'),  
407 (13, 'Business License', 'https://storage.example.com/biz13.jpg', 'Verified', '2024-01-10');
```

TEST QUERY & RESULT:

```
1 SELECT * FROM Verification;
```

Result Grid Filter Rows: Export: Wrap Cell Content:										
	VerificationID	UserID	DocumentType	DocumentURL	Status	VerifiedBy	VerifiedDate	Notes	CreatedAt	UpdatedAt
▶	1	1	Passport	https://storage.example.com/passport1.jpg	Verified	NULL	2024-01-15	NULL	2026-03-19 06:42:38	2026-03-19 06:42:38
	2	2	Drivers License	https://storage.example.com/dl2.jpg	Verified	NULL	2024-01-20	NULL	2026-03-19 06:42:38	2026-03-19 06:42:38
	3	6	Passport	https://storage.example.com/passport6.jpg	Verified	NULL	2024-02-01	NULL	2026-03-19 06:42:38	2026-03-19 06:42:38
	4	7	ID Card	https://storage.example.com/id7.jpg	Verified	NULL	2024-02-05	NULL	2026-03-19 06:42:38	2026-03-19 06:42:38
	5	13	Business License	https://storage.example.com/biz13.jpg	Verified	NULL	2024-01-10	NULL	2026-03-19 06:42:38	2026-03-19 06:42:38

TRIPLE RELATIONSHIP TESTS

RELATIONSHIP 1: Booking – Guest - Property

```
1 • SELECT b.BookingID,  
2         CONCAT(u.FirstName, ' ', u.LastName) AS GuestName,  
3         p.Title AS Property,  
4         bs.StatusName AS BookingStatus  
5 FROM Booking b  
6 JOIN User u ON b.GuestID = u.UserID  
7 JOIN Property p ON b.PropertyID = p.PropertyID  
8 JOIN BookingStatus bs ON b.StatusID = bs.StatusID  
9 LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

BookingID	GuestName	Property	BookingStatus
1	John Smith	Cozy Downtown Apartment	Confirmed
2	Emma Johnson	Beachfront Villa	Confirmed
3	Michael Brown	Mountain Retreat Cabin	Confirmed
4	Sarah Davis	City Center Loft	Confirmed
5	David Wilson	Lakeside Condo	Confirmed

RELATIONSHIP 2: Reviews between Users

```
1 • SELECT r.ReviewID,  
2         CONCAT(rev.FirstName, ' ', rev.LastName) AS Reviewer,  
3         CONCAT(ree.FirstName, ' ', ree.LastName) AS Reviewee,  
4         r.Rating, r.Comment  
5 FROM Review r  
6 JOIN User rev ON r.ReviewerID = rev.UserID  
7 JOIN User ree ON r.RevieweeID = ree.UserID  
8 WHERE r.ReviewerID != r.RevieweeID  
9 LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

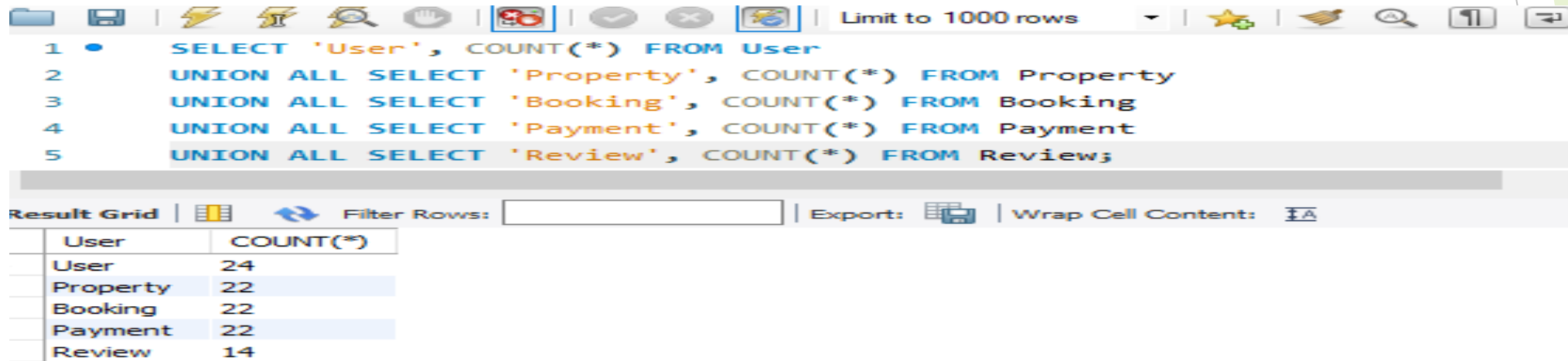
ReviewID	Reviewer	Reviewee	Rating	Comment
1	John Smith	Robert Miller	5	Great apartment, very clean and well located!
2	Emma Johnson	Jennifer Taylor	4	Beautiful villa, but WiFi was slow.
3	Michael Brown	William Anderson	5	Perfect mountain getaway!
4	Sarah Davis	Linda Thomas	4	Cool loft, amazing design.
5	David Wilson	James Jackson	5	Best lake view ever!

SUMMARY

PROJECT COMPLETION SUMMARY

Tables Created: 22
Views Created: 3
Stored Procedures: 2
Triggers: 3
Functions: 2

SAMPLE DATA COUNTS & RESULT



The screenshot shows a database query editor interface. The top toolbar includes icons for file operations, query execution, and a 'Limit to 1000 rows' dropdown. The SQL query is as follows:

```
1 SELECT 'User', COUNT(*) FROM User
2 UNION ALL SELECT 'Property', COUNT(*) FROM Property
3 UNION ALL SELECT 'Booking', COUNT(*) FROM Booking
4 UNION ALL SELECT 'Payment', COUNT(*) FROM Payment
5 UNION ALL SELECT 'Review', COUNT(*) FROM Review;
```

Below the query, the 'Result Grid' is displayed with the following data:

User	COUNT(*)
User	24
Property	22
Booking	22
Payment	22
Review	14

KEY ACHIEVEMENT:

3 Triple Relationships Implemented
2 Recursive Relationships Implemented
20+ Sample Rows per Main Table
All Foreign Key Constraints Working
Views, Procedures, Triggers, Functions Created